

个人作业报告

题目：**并行程序设计与算法**

理论课第二次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	吴迪

2026 年 4 月 16 日

【摘　要】

此作业报告是并行程序设计与算法课程的**第二次作业**，作业题目来自智算习堂[1]的作业内容。

关键词：　并行程序设计与算法

目录

1 第一题 1
1.1 题目 1
1.2 答案 1

2 第二题 3
2.1 题目 3
2.2 答案 3

3 第三题 5
3.1 题目 5
3.2 答案 5

4 第四题 7
4.1 题目 7
4.2 答案 7

5 第五题 9
5.1 题目 9
5.2 答案 9

参考文献 11

1 第一题

1.1 题目

死锁是并行程序中的典型错误之一，以下代码段实现了在两个进程上交换各自长度为 N 的浮点数组的功能。

```
float send_buf[N], recv_buf[N];
if(my_rank== 0){ //进程 0
    MPI_Send(send_buf, N, MPI_FLOAT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(recv_buf, N, MPI_FLOAT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE)
    ;
} else if(my_rank== 1){ //进程 1
    MPI_Send(send_buf, N, MPI_FLOAT, 0, 1, MPI_COMM_WORLD);
    MPI_Recv(recv_buf, N, MPI_FLOAT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE)
    ;
}
```

1) 分析该程序为什么在 N 较小时能够成功运行，而当 N 的取值非常大时，有可能永久挂起。
2) 请给出一种可行的解决方案并简要说明。

1.2 答案

1.2.1 （1）原因分析

当两个进程都先调用阻塞式发送 `MPI_Send` 而未先发布对应的接收 `MPI_Recv` 时，若消息大小 N 超过 MPI 实现内部的"eager" 缓冲阈值，发送操作会阻塞等待对端接收；双方都在 `MPI_Send` 上等待，从而发生死锁。对于较小的 N，MPI 常使用 eager 协议把数据复制到内部缓冲区使发送立即返回，因此看起来没有问题；但当 N 很大时，发送不会被缓冲而会阻塞，进而可能导致永久挂起。

1.2.2 （2）可行的解决方案

死锁的根因是阻塞发送与缓冲策略，解决方法是使用可避免互相等待的通信模式（`MPI_Sendrecv`、非阻塞或先贴接收）。

1.2.2.1 使用原子式的发送/接收函数 MPI_Sendrecv：

该调用在内部避免了双方同时阻塞，能可靠交换任意大小的数据（由实现保证）。

```
int peer = 1 - my_rank;
MPI_Sendrecv(send_buf, N, MPI_FLOAT, peer, 0,
             recv_buf, N, MPI_FLOAT, peer, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

1.2.2.2 使用非阻塞通信先贴接收再发送：

先发布接收可以保证对端的发送不会无限等待，非阻塞发送/接收配合等待也能避免死锁。

```
MPI_Request reqs[2];
MPI_Irecv(recv_buf, N, MPI_FLOAT, peer, 0, MPI_COMM_WORLD, &reqs[0]);
MPI_Isend(send_buf, N, MPI_FLOAT, peer, 0, MPI_COMM_WORLD, &reqs[1]);
MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
```

1.2.2.3 改变顺序：

一端先执行MPI_Recv再MPI_Send,另一端相反。或者,使用MPI_Isend/MPI_Irecv的混合策略以保证不存在循环等待。

2 第二题

2.1 题目

在分布式算法中，有时我们需要实现类似于 MPI_Allgather 的功能，即每个进程都有一个局部值，最后每个进程都要获得所有进程的值，实现全局数据同步。使用 C 语言编写一段符合以下要求的程序：

1) 在一个包含 size 个进程的环形传递结构（即进程 q 总是将消息发送给 (q+1) mod P，并从 (q-1+P) mod P 接收消息，其中 P 为进程总数）中，实现 2 中的要求。
2) 每个进程持有一个整数 my_val。经过若干步迭代，每个进程计算出环内所有值的累加和。
3) 使用 MPI_Sendrecv 以避免死锁。

2.2 答案

下面给出一种在环形结构上使用 MPI_Sendrecv 的实现思路和示例代码：每个进程将自己的值沿环传递，经过 $P-1$ 步后，每个进程都能收到环上所有进程的值并累加。

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int my_val;
    my_val = rank + 1;

    int sum = my_val;
    int send_val = my_val;
    int recv_val;
```

```
    int right = (rank + 1) % size;
    int left  = (rank - 1 + size) % size;

    for (int step = 1; step < size; ++step) {
        MPI_Sendrecv(&send_val, 1, MPI_INT, right, 0,
                     &recv_val, 1, MPI_INT, left,  0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        sum += recv_val;
        send_val = recv_val;
    }

    printf("rank %d: sum = %d\n", rank, sum);

    MPI_Finalize();
    return 0;
}
```

说明：每个进程在第 i 步发送上一步收到的值（第 0 步发送自己的值），经过 $P-1$ 步后，环上所有值都已被遍历并累加。使用 `MPI_Sendrecv` 可以在一次调用内完成发送和接收，避免了阻塞发送引起的死锁。

3 第三题

3.1 题目

有两个长度为 ($n = 12$) 的整数向量 x 和 y，其初值如下：

$$x = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23]$$

$$y = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]$$

现使用 `comm_sz= 4` 个 MPI 进程完成向量加法，要求如下：

1) 由 0 号进程初始化向量 x 和 y；
2) 使用 MPI 通信操作，将两个向量平均分发给各个进程；
3) 每个进程计算自己负责部分的局部和；
4) 再将结果收集到 0 号进程；
5) 最后由 0 号进程输出结果向量 z。

请回答下列问题：

1) 每个进程分别处理向量中的哪几个元素？
2) 最终结果向量 z 是什么？
3) 写出完成该任务时需要用到的主要 MPI 函数名称，并简要说明它们的作用。

3.2 答案

1) 每个进程分别处理的元素（下标按从 0 开始计数，向量长度 $n = 12$，进程数 $P = 4$，每个进程处理 $n/P = 3$ 个元素）：
   - 进程 0: 元素下标 $0, 1, 2$ （对应 $x = [1, 3, 5]$, $y = [2, 4, 6]$）
   - 进程 1: 元素下标 $3, 4, 5$ （对应 $x = [7, 9, 11]$, $y = [8, 10, 12]$）
   - 进程 2: 元素下标 $6, 7, 8$ （对应 $x = [13, 15, 17]$, $y = [14, 16, 18]$）
   - 进程 3: 元素下标 $9, 10, 11$（对应 $x = [19, 21, 23]$, $y = [20, 22, 24]$）
2) 最终结果向量 z（逐元素相加）为：

$$z = [3, 7, 11, 15, 19, 23, 27, 31, 35, 39, 43, 47].$$

3) 完成该任务常用的主要 MPI 函数及作用：

- `MPI_Init`: 初始化 MPI 环境。
- `MPI_Comm_size`: 获取进程总数（P）。
- `MPI_Comm_rank`: 获取当前进程的秩（rank）。
- `MPI_Scatter`: 将根进程上的数组按等份分发到各个进程（每个进程接收相同数量的数据）。
- `MPI_Gather`: 将各进程计算出的局部结果收集到根进程。
- `MPI_Finalize`: 终止 MPI 环境。

若数据块大小不一致，应使用 `MPI_Scatterv`/`MPI_Gatherv` 或手工发送/接收。

4 第四题

4.1 题目

设有一个长度为 ($n = 10$) 的整数数组 a：

$$a = [5, 8, 2, 6, 1, 9, 4, 7, 3, 10]$$

现在希望使用 comm_sz= 4 个 MPI 进程对该数组中的元素分别乘以 2，最后再把结果收集回 0 号进程。

已知 MPI_Scatter 和 MPI_Gather 的一个限制是：每个进程接收或发送的数据个数必须相同。

请回答下列问题：

1) 为什么在本题中不能直接使用一次 MPI_Scatter 和一次 MPI_Gather 完成任务？
2) 如果采用“前 2 个进程各处理 3 个元素，后 2 个进程各处理 2 个元素”的分配方案，请写出各进程负责的数据范围。
3) 给出一种可行的修改思路，使程序在 n 不能被 comm_sz 整除时仍能正确完成计算。

4.2 答案

1) 不能直接使用一次 MPI_Scatter 和一次 MPI_Gather 的原因：这两个函数要求每个进程发送/接收的数据个数相同，但本题 $n = 10$、进程数 $P = 4$ 时 10 不能被 4 整除，无法在一次等份划分中保证每个进程接收相同数量的数据。因此必须使用变长的分发/收集或额外处理。
2) 若采用“前 2 个进程各处理 3 个元素，后 2 个进程各处理 2 个元素”的分配方案，则各进程负责的数据范围（按 0 起始下标）：
   - 进程 0: 下标 $0, 1, 2$（对应元素 $[5, 8, 2]$）
   - 进程 1: 下标 $3, 4, 5$（对应元素 $[6, 1, 9]$）
   - 进程 2: 下标 $6, 7$（对应元素 $[4, 7]$）
   - 进程 3: 下标 $8, 9$（对应元素 $[3, 10]$）

3) 根进程（例如 rank 0）准备原始数组 sendbuf，并计算好 counts 与 displs；每个进程根据 counts[rank] 分配接收缓冲区 recvbuf，调用 MPI_Scatterv 分发局部数据，做本地计算后再用 MPI_Gatherv 收集回根进程。对任意长度和进程数，可用如下方法计算 counts 和 displs：

   - 令 $q = \lfloor n/P \rfloor$，$r = n \bmod P$，则对每个进程 i，counts[i]=q+(i<r?1:0)。
   - displs[0]=0；对于 $i > 0$，displs[i]=displs[i-1]+counts[i-1]；

   示例代码片段：

```
int counts[4] = {3,3,2,2};
int displs[4] = {0,3,6,8};
int rank;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
int recvcount = counts[rank];
int *recvbuf = malloc(recvcount * sizeof(int));
// 根进程上有 int sendbuf[n]; 其它进程该参数可置为 NULL
MPI_Scatterv(sendbuf, counts, displs, MPI_INT,
             recvbuf, recvcount, MPI_INT, 0, MPI_COMM_WORLD);
// 本地计算：每个元素乘以 2
for (int i = 0; i < recvcount; ++i) recvbuf[i] *= 2;
// 收集回根进程
MPI_Gatherv(recvbuf, recvcount, MPI_INT,
            result, counts, displs, MPI_INT, 0, MPI_COMM_WORLD);
free(recvbuf);

```

   MPI_Scatterv 和 MPI_Gatherv 在根进程上读取 counts 与 displs，非根进程这两个数组参数会被忽略；但各进程必须知道自己的接收长度（recvcount）以便正确分配接收缓冲区。该方法通用且避免了对数据填充或多次点对点发送的手工实现。

5 第五题

5.1 题目

考虑使用 MPI 实现数值积分的梯形法则，假设函数 $f(x)$ 在区间 $[ab]$ 上是连续的，我们将区间划分成 n 个等宽的小梯形，每个梯形的宽度为 $\hbar = \frac{b-a}{n}$，积分的近似公式为：

$$\int_a^b f(x)dx \approx \hbar \left[\frac{f(a)}{2} + \sum_{i=1}^{n-1} f(a + i * \hbar) + \frac{f(b)}{2} \right]$$

现在你编写了一个 MPI 程序，使用 p 个进程来并行计算该积分，每个进程负责计算一部分梯形的面积，然后通过 `MPI_Reduce` 将局部结果汇总到进程 0，最后由进程 0 输出总积分值。

1) 假设 n 能被 p 整除，请写出每个进程负责的梯形数量、起始点和结束点的计算公式。如果 n 不能被 p 整除，每个进程负责的梯形数量又是多少。
2) 使用 `MPI_Reduce` 汇总结果，该集合通信的时间复杂度是多少，假设使用的是树形通信结构。
3) 假设串行程序的运行时间为 $T_{serial} = k*n$，并行程序的运行时间为：$T_{parallel}(n,p) = \frac{k*n}{p} + c * \log_2(p)$，其中 c 为常数，请推导出加速比 $S(n,p)$ 和效率 $E(n,p)$ 的表达式。当 n 固定，p 增大时，效率会如何变化，为什么。

5.2 答案

1) **当 n 能被 p 整除时：** 每个进程负责的梯形数量为

$$n_{local} = \frac{n}{p}.$$

对于秩为 rank 的进程，其负责的区间为

$$x_{start} = a + \text{rank} \cdot n_{local} \cdot \hbar, \qquad x_{end} = a + (\text{rank} + 1) \cdot n_{local} \cdot \hbar,$$

其中 $\hbar = (b-a)/n$。

如果 n 不能被 p 整除：可令

$$q = \left\lfloor \frac{n}{p} \right\rfloor, \qquad r = n \bmod p,$$

$$n_{local}(\text{rank}) = q + (\text{rank} < r ? 1 : 0),$$

即前 r 个进程多分配 1 个梯形。

2) 使用树形归约（树形通信结构）的 `MPI_Reduce` 的时间复杂度为 $O(\log p)$。若要更精确地包含延迟项和带宽项，通常写作 $O(\alpha \log p + \beta m \log p)$。若只关心步数为 $\Theta(\log p)$。

3) 给定

$$T_{serial} = k\,n, \qquad T_{parallel}(n,p) = \frac{k\,n}{p} + c\,\log_2 p,$$

则加速比为

$$S(n,p) = \frac{T_{serial}}{T_{parallel}} = \frac{k\,n}{\dfrac{k\,n}{p} + c\,\log_2 p} = \frac{k\,n\,p}{k\,n + c\,p\,\log_2 p}.$$

效率为

$$E(n,p) = \frac{S(n,p)}{p} = \frac{k\,n}{k\,n + c\,p\,\log_2 p} = \frac{1}{1 + \dfrac{c\,p\,\log_2 p}{k\,n}}.$$

当固定 n，p 增大时，分母中通信项 $\frac{c\,p\,\log_2 p}{k\,n}$ 会增长，导致效率下降；直观上，通信开销随进程数增加（且近似为 $p \log p$ 级别）增长，最终通信会掩盖并行计算的收益，效率趋向 0。

参考文献

[1] [Z]. https://easyai.sysu.edu.cn.